ASHESI UNIVERSITY

Technical REPORT

LAB1, Intro to AI Robotics

System Overview:

- The goal was to build a ROS2 system that processes simulated distance sensor data and triggers safety alerts...

- An ROS 2 system is made up of smaller, independent programs called "nodes" that run simultaneously and communicate with each other.

- Indeed, my system consists of three distinct nodes, a custom interface, and is heavily driven by the principle of Test-Driven Development (TDD)...

Node Architecture and Custom Interfaces:

- Sensor Node: This node acts as a publisher; it simulates a hardware sensor and regularly broadcasts distance readings in meters to a particular 'topic'...

- Processing Node: This node acts as both a subscriber and a publisher. It listens to the sensor's topic, applies the specific processing logic (like converting the meters to centimeters, for example), and then publishes that newly transformed data to a different topic. This modular publish/subscribe system is effective because the sensor node doesn't need to care what happens to its data after it broadcasts it!

- Service Node: This node acts as a Service server and a subscriber. While topics are for constant data streams, services are for single request/reply communications. A custom interface package (distance_sensor_interfaces) was created, containing SetThreshold.srv to safely separate the interface definitions from the python logic... The node subscribes to the sensor readings and uses ROS2's built-in logging system to trigger a warning message if the distance drops below the safe threshold defined by the service.

System Configuration and Launching:

- ★ The system was made to be dynamic and easy to start without changing the underlying Python code.

- ★ Parameter File (params.yaml): Parameters allow developers to change the behavior of a node on the fly. Variables like the publishing frequency and the default safety threshold were mapped into a single YAML configuration file, allowing for easy customization.

- ★ Launch File (system_launch.py): In truth, manually starting three nodes and loading parameters from different terminals is tedious. Herein comes the awesome python launch script (system_launch.py) to automate the bring-up process, simultaneously booting the sensor, processing, and service nodes while dynamically passing my parameter file into them...

<u>Test-Driven Development (TDD) principle:</u>

The TDD principle requires one to write tests before writing the actual feature code...

PyTest unit tests were written that initially failed because the nodes didn't exist yet (Red Phase)

-Minimal Python code required to initialize the nodes and pass the assertions...that is, the 'Green Phase'

*A completely clean test suite (0 failures) was achieved before moving on.

While it does require more initial setup, writing tests first force me to clearly understand the design requirements before coding and prevent breaking existing features as the system grows.

Code Quality and Package Organization:

- ✓ Project was separated into two distinct packages: distance_sensor_system for the Python scripts and distance_sensor_interfaces for the custom .srv file. This is a ROS2 best practice that prevents dangerous circular dependencies between packages...

- ✓ The colcon build tool was also utilized to compile the workspace, and files were organized strictly into standard folders like src, test, launch, and config (configuration)...